Adaptive Software-Defined Perimeter Placement for Dynamic User Distributions in Zero-Trust Networks

Xuanbo Huang, *Member, IEEE*, Kaiping Xue, *Senior Member, IEEE*, Lutong Chen, *Member, IEEE*, Zixu Huang, *Graduate Student Member, IEEE*, Jiangping Han, *Member, IEEE*, Jian Li, *Senior Member, IEEE*, and Ruidong Li, *Senior Member, IEEE*

***Abstract*—Zero Trust Network (ZTN) has become a widely adopted security architecture in enterprise and campus networks. A typical implementation involves Software-Defined Perimeter (SDP), which deploys proxy-based gateways to mediate user-service communication and block unauthorized access. However, existing deployment approaches rely on empirical placement and static user binding, overlooking user mobility and dynamic traffic patterns. This limitation can lead to suboptimal connection paths, increased delays, and potential performance bottlenecks. This paper presents the first formulation of the SDP gateway placement problem under user mobility. We model this challenge as a multiobjective optimization problem that jointly minimizes user-server connection delay and balances load across gateways. To address it, we propose a two-stage solution comprising offline deployment and online adaptation. The offline stage introduces *PathHeatMap-guided SDP embedding*, which aggregates user-server flow paths into a heatmap to identify convergence hotspots and guide SDP placement at high-impact nodes. The online stage integrates *mobility tracking* and *deferred SDP rebinding*, which dynamically responds to user mobility by temporarily assigning displaced users to nearby gateways and triggering global reassignment only when necessary. Extensive simulations demonstrate that our method attains near-Pareto-front delay and load-balance trade-offs while running orders of magnitude faster than heavy solvers (MILP, Convex, NSGA-II).**

***Index Terms*—Zero trust network, software-defined perimeter, adaptive placement, delay optimization, load-balancing.**

I. Introduction

ZERO-TRUST network (ZTN) [1] adopts the principle of "never trust, always verify," assuming potential threats are located both inside and outside a network. To mitigate such threats, ZTNs enforce continuous authentication, authorization, and monitoring for every user, device, and application, regardless of their locations. Initially introduced in 2011, ZTN has gained increasing relevance. According to Grand View Research [2], the global ZTN market was valued at $24.84 billion in 2022 and is projected to grow at a compound annual rate of 16.6% through 2030. Furthermore, Gartner reports that 63% of organizations worldwide have fully or partially adopted zero trust architecture [3].

Several approaches can be used to implement ZTN, such as micro-segmentation and identity-based access control. Among them, the Software-Defined Perimeter (SDP) model has gained traction for its flexibility, scalability, and ease of integration with existing infrastructure [4], [5], [6], [7], [8]. SDP establishes a dynamic perimeter by introducing proxy-based intermediaries, SDP gateways, which mediate all communication between users and services. SDP offers a practical pathway for legacy networks to transition toward zero-trust networks. Operators can deploy SDP gateways as middleboxes to proxy user-service traffic, enhancing security while staying transparent to applications.

The SDP gateway placement directly shapes user-perceived performance: poorly placed gateways lengthen propagation paths and concentrate traffic, degrading Quality of Experience (QoE) in practice [4], [6]. The effect is amplified in large enterprise environments, where user locations vary substantially over time with unpredictable spatiotemporal patterns [9], [10]. Static deployments often fail to keep pace with these dynamics, motivating methods that adapt placement and binding to current network conditions.

Related efforts on Zero Trust deployment in emerging domains (e.g., 6G [11], [12], [13] and industrial control systems [14], [15], [16]) are typically tailored to specific architectures and objectives, which limits generality. Within enterprise settings, prior placement studies [4], [5] largely rely on empirical heuristics.

Placement methods in adjacent areas such as Software-Defined Networking (SDN), Network Functions Virtualization (NFV), and Mobile Edge Computing (MEC) often employ heavy optimizers, including ILP [17], convex relaxations [18], and multi-objective meta-heuristics such as NSGA-II [19]. These approaches demonstrate feasibility, but their assumptions and computational footprints complicate online operation under mobility. The practical problem, therefore, is to adapt to user mobility without predicting it or relying on specific mobility assumptions. In addition, the solution must react at operational timescales and balance multiple objectives. Mobility is stochastic and nonstationary, making prediction-based placements brittle in

Received 19 May 2025; revised 12 November 2025; accepted 8 December 2025. Date of publication 31 December 2025; date of current version 19 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62201540, Grant 62302472, and Grant 62372425 and in part by the Youth Innovation Promotion Association of Chinese Academy of Sciences under Grant Y202093. Recommended for acceptance by Dr. Yuan Shen. *(Corresponding author: Kaiping Xue.)*

Xuanbo Huang, Kaiping Xue, Zixu Huang, Jiangping Han, and Jian Li are with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230027, China (e-mail: kpxue@ustc.edu.cn).

Lutong Chen is with the Network and Information Center, University of Science and Technology of China, Hefei 230026, China.

Ruidong Li is with the Institute of Science and Engineering, Kanazawa University, Kakuma 920-1192, Japan.

Digital Object Identifier 10.1109/TNSE.2025.3649680

deployment. Online adaptation is viable only if the algorithm is lightweight and scales smoothly with network size. At the same time, operators must manage competing goals: controlling end-to-end delay and reducing load variability, while keeping rebinding overhead bounded.

In this paper, we design an adaptive SDP placement architecture for large-scale networks that considers user mobility. We begin by formulating the SDP placement problem as a multi-objective optimization task that minimizes user-server connection delay while maintaining balanced load across deployed SDP gateways. We prove that the problem can be reduced to a variant of the NP-hard set cover problem. To solve this, we design a two-stage adaptive framework that integrates static deployment with dynamic reassignment.

The proposed framework consists of an offline deployment stage and an online binding stage. In the offline stage, we introduce a flow aggregation mechanism called *PathHeatMap-guided SDP embedding*, which constructs a heatmap of user-server path overlaps to guide the placement of SDPs at high-impact nodes. This mechanism prioritizes direct flow capture by identifying convergence hotspots in the network topology. For the flows not covered by on-path SDPs, the system performs a constrained fallback assignment to minimize detour overhead. In the online stage, we design a method named *deferred SDP re-binding*, which reacts to user mobility by temporarily assigning displaced users to nearby SDPs and triggering global rebinding only when necessary. This approach enables a fast response to movement without frequent global recomputation.

We evaluate the framework on five real L3 topologies from the Internet Topology Zoo [20] with varied user, service, and proxy counts. We compare against five principal baselines: Random, Gateway (nearest assignment), Dynamic Program, Convex relaxation, Mixed-Integer Linear Programming (MILP), and an NSGA-II to trace empirical Pareto fronts. To study mobility using real traces, we map the GeoLife [21], [22] dataset (182 users) onto the Chinanet topology, and we leverage a standard FreeRADIUS [23] to log association events and extract user mobility tables, providing prediction-free snapshots of user locations. Our experiments cover: (i) Pareto trade-offs between delay and load variability, (ii) QoE objective performance under different situations, (iii) scalability of runtime with topology size and user count, (iv) authentication-delay impact, and (v) a threshold ablation that quantifies the quality–overhead knee for online rebinding.

To summarize, this paper makes the following main contributions:

1) We present a practical SDP placement and rebinding framework that integrates with existing Zero Trust infrastructure. It leverages readily available RADIUS association events to construct per-epoch snapshots of user locations and supports online rebinding via a simple threshold rule, enabling prediction-free adaptation to mobility with minimal engineering overhead.
2) We develop a *PathHeatMap*-guided proxy embedding method for offline deployment and a deferred SDP rebinding mechanism for online adaptation. The offline solver optimizes a scalarized objective that balances end-to-end delay and load variability, yielding a lightweight and efficient solution, attaining Pareto-front quality on tractable instances while avoiding the computational burden of MILP, convex relaxations, and NSGA-II.
3) We conduct an extensive evaluation on five real L3 topologies against five baselines (Random, Gateway, Dynamic Programming, Convex, MILP), and include NSGA-II to trace empirical Pareto fronts. Results demonstrate that our solutions cluster at the low-delay and low-CV knee, run orders of magnitude faster than heavy solvers, and scale smoothly with network size and user count.

The remainder of this paper is structured as follows: Section II discusses the necessary background on ZTN, SDP, and related work. Section III formulates the problem and proves its NP-hardness. In Section IV, we elaborate on the proposed architecture and algorithms. Section V presents the simulation results. Section VI discusses the limitations of this work. Finally, we conclude the proposed work in Section VII.

II. Background and Related Work

To better compare the proposed work within the broader research landscape, Table I presents a comparative summary of representative studies across two categories: First, the security-driven designs in ZTN architectures, including authentication, access control, integration mechanisms, and SDP-based architectures. Second, placement methods span different environments and methodological classes. The table records each work's treatment of mobility modeling, placement objectives, delay minimization, load balancing, and migration overhead, as well as the computational footprint.

Our approach complements the ZTN security literature, which focuses on upper-layer functionality, whereas we investigate where to place gateways and how to rebind them. It also differs from prior placement studies in three key ways. First, in this paper, mobility is explicit and prediction-free: rather than learning a model, we operate on observed user locations to adapt online. Second, the objective is tailored to SDP's role as an in-path proxy, balancing end-to-end delay and load variability, rather than optimizing service migration or complex SFC coupling. Third, the migration cost is negligible for SDP gateways, which maintain only a lightweight state. Accordingly, our design emphasizes a lightweight algorithm suitable for large networks where heavy solvers or deep learners are impractical.

A. Zero Trust Networks

For decades, traditional network architectures relied on a clear demarcation between trusted internal networks and untrusted external environments. Security was enforced through perimeter-based defenses, such as firewalls and gateways, a method often referred to as the "castle-and-moat" model. However, this approach has proven insufficient in modern network environments characterized by increasing mobile and Internet of Things (IoT) traffic and distributed workforces.

To address these limitations, the ZTN paradigm was introduced in 2010 and has been extensively studied [1]. ZTN eliminates the distinction between internal and external users,

TABLE I
COMPARISON OF RELATED WORK WITH OUR APPROACH

Category	Prior Works	User Mobility	Placement	Delay Minimization	Load Balancing	Migration Cost
ZTN Architecture	[11]–[13], [24], [25]	Static	Not discussed	Not discussed	Not discussed	Not discussed
ZTN Authentication	[16], [26]–[28]	Not applicable	Not applicable	Not considered	Not considered	Not discussed
ZTN Access Control	[29]–[32]	Not applicable	Not applicable	Not applicable	Not applicable	Not applicable
ZTN Framework	[14], [33]–[39]	Static	Empirical	Not considered	Not considered	Ignored*
MEC Placement	[40]–[48]	Static/Dynamic	Heuristic/ML	Considered	Often skipped	Considered
SDN/NFV Placement	[49]–[55]	Static	Heuristic/ML	Considered	Considered	Considered
SDP Architecture	[4]–[8]	Static	Empirical	Increased latency	Not addressed	Ignored*
Our Work	–	**Considered**	**Adaptive placement**	**Delay minimized**	**Load Balanced**	**Ignored***

* Unlike stateful services in MEC/SDN, SDP gateways are stateless. Thus, migration cost is negligible and excluded from our optimization.

requiring continuous authentication, authorization, and context-aware access control. A key milestone in ZTN development was Google's introduction of the BeyondCorp architecture in 2014 [56], which enables secure, location-independent access to services. In this model, applications are hosted in the cloud, and access is mediated through an identity-aware proxy that evaluates device security posture and user credentials before granting access. Then, Bring Your Own Device (BYOD) has emerged as a popular architecture, enabling users to access corporate resources using personal devices. ZTN models are particularly well-suited to address the security challenges introduced by BYOD by enforcing continuous verification [57], [58].

Recently, the ZTN model has been widely adopted across a range of domains, illustrating its versatility and adaptability in emerging networking paradigms. In sixth-generation (6G) networks, Chen et al. [13] proposed a software-defined ZTN architecture that enforces secure access control through adaptive collaboration among control domains. This design mitigates a variety of security threats, including Distributed Denial-of-Service (DDoS) attacks, malware propagation, and zero-day exploits. Expanding upon this foundation, Sedjelmaci et al. [12] investigated the integration of zero trust architecture (ZTA) with artificial intelligence (AI) to enhance 6G network security. They introduced a hierarchical defense framework that leverages adaptive AI algorithms executed by distributed agents to protect both infrastructure and services. In a complementary study, Sedjelmaci et al. [24] developed a ZTA-enabled intrusion detection system tailored for 6G edge computing. Da Silva et al. [25] explored the integration of Zero-Trust principles with 6G URLLC to enhance Federated Learning (FL) performance in vehicular networks, demonstrating significant improvements in latency, throughput, and model robustness. Beyond 6G, Liu et al. [11] proposed a ZTN-based mobile network security architecture that integrates authentication, access control, and network slicing to defend against lateral movement and control-plane attacks in operator environments.

Software-Defined Perimeter: SDP has emerged as a promising architecture for realizing ZTN principles [4], [5], [6], [7]. An SDP system typically comprises two key components: a controller that manages user authentication, authorization, and policy enforcement. Gateways that serve as secure intermediaries between authenticated users and protected services. Upon successful authentication, users are dynamically assigned to SDP gateways, which establish secure communication paths and conceal internal infrastructure from unauthorized entities, effectively preventing lateral movement within the network.

Several prior studies have examined SDP system design and deployment methods. To illustrate, Moubayed et al. [6] proposed a client-gateway model in which SDP gateways are deployed at the network edge to enforce internal access control. Kumar et al. [4] investigated SDP's role in mitigating DDoS attacks. Their results demonstrate that SDP significantly reduces the attack surface by obscuring service endpoints and enforcing strict access policies. Nevertheless, their study omits the discussion of gateway deployment methods and their impact on service quality or responsiveness under varying user mobility conditions. To enhance deployment flexibility, Singh et al. [5] introduced an NFV-based SDP framework leveraging virtualization to support multi-level access control and dynamic policy updates. Despite its architectural scalability, this work does not address how gateway placement affects end-to-end latency or load balancing, particularly in mobile or high-density user environments. In mobile networks, Bello et al. [7] proposed vEPC-vSDP, a framework that integrates SDP into 5G core networks. Abdelhay et al. [8] introduced an SDP-based VPN alternative for 6G cores, incorporating Moving Target Defense (MTD) to enhance authentication resilience. While these works advance architectural flexibility, they do not examine the placement of SDP gateways or consider spatial-temporal user dynamics, which are critical in mobile or high-density settings. Consequently, their evaluations reveal measurable delay overheads and potential performance bottlenecks.

ZTN Security: The security mechanisms of ZTN have been extensively studied across various domains. In this section, we discuss recent advances in authentication techniques, access control methods, and architectural frameworks.

In the area of authentication, recent efforts have explored behavior-aware and AI-assisted mechanisms that dynamically evaluate user and device trustworthiness. Nagarajan et al. [16] proposed an AI-based ZTN framework for industrial applications, leveraging contextual trust evaluation and predictive modeling to strengthen identity verification and reduce dependency on static credentials. Similarly, Zhang et al. [26] introduced a behavior-based dynamic authentication scheme that enhances identity assurance by integrating user behavior profiles into the authentication process. Meng et al. [27] proposed a blockchain-based continuous device-to-device authentication protocol that removes reliance on trusted authorities by leveraging Byzantine fault tolerance for secure key generation under a zero-trust

architecture. Jing et al. [28] proposed a zero-trust authentication mechanism for IoT networks that integrates radio frequency fingerprint identification to enhance security in edge network environments.

From an architectural perspective, ZTN principles have been integrated into diverse system-level contexts. Feng et al. [14] developed a cyber-physical zero-trust framework for industrial systems, emphasizing continuous verification and identity enforcement across heterogeneous cyber-physical layers. Claudio et al. [33] introduced a zero-trust-inspired security architecture for IIoT environments, combining SDN-based microsegmentation with a centralized management layer to support decentralized, real-time operations. Shen et al. [34] proposed a zero-trust framework for enterprise endpoint security that integrates identity management, policy enforcement, and situational awareness to control user access. Zero Trust models have also been applied to specialized domains. Hong et al. [35] presented SysFlow, a programmable framework for dynamic policy enforcement. Zhu et al. [36] proposed a blockchain-based architecture for secure healthcare communication, while Dubin et al. [37] introduced DeepCDR for zero-trust content sanitization. Liu et al. [38] addressed cross-domain data sharing by proposing a scalable ZTN framework based on checkable encryption and sharded blockchains for cloud-edge-end systems. In a related effort, Liu et al. [39] developed a blockchain-enabled information-sharing framework for zero-trust IoT environments, achieving both data privacy and trust assurance through smart contracts.

Access control in ZTN has evolved to incorporate adaptive, game-theoretic, and learning-based methodologies. Bello et al. [29] presented a dynamic zoning framework that applies stochastic games and meta-learning to enforce access policies under uncertainty, enabling adaptive and threat-responsive control. Ge et al. [30] proposed GAZETA, a game-theoretic authentication framework that mitigates lateral movement in 5G IoT networks by modeling attacker-defender interactions and enforcing resilient, zero-trust access policies. Routray et al. [31] introduced a zero-trust access control framework for Industrial IoT environments that leverages ciphertext-policy attribute-based encryption (CP-ABE) to enable fine-grained and dynamic authorization. Mukta et al. [32] proposed a blockchain-based access control delegation model for large-scale systems, integrating self-sovereign identity for authentication and a sanitizable signature scheme to enable flexible access control.

B. Placement Optimization

Placement optimization has been widely studied across MEC and SDN/NFV. Methods include exact and convex optimization, combinatorial heuristics, evolutionary or other metaheuristics, and learning-based approaches [40], [59], [60], [61], [62]. We organize the discussion along two axes: operating environment (wireless edge, datacenter or cloud, ISP-scale L3) and methodological class (exact or convex, heuristic or combinatorial, evolutionary, learning or MDP). While these works share the goal of improving QoS, architectural and operational differences across domains lead to distinct formulations and assumptions. Below, we review representative advances with an eye to their relevance for SDP placement under mobility.

TABLE II
COMPARISON WITH MAINSTREAM PLACEMENT METHODS

Method Class	Scalability	Runtime (Chinanet)	Online Deployment	Dominated by NSGA-II front?
Nearest	Linear	0.18s	✓	✓
Random	Linear	0.13s	✓	✓
DP	Superlinear	8.48s	×	✓
Convex	Superlinear	8573.55s	×	✓
MILP	Superlinear	8622.47s	×	✓
NSGA-II	Superlinear	7684.28s	×	-
Proposed	**Linear**	**2.28s**	✓	**Not dominated (knee region)**

Note: Convex and MILP are solved with standard solvers CVX [18] and Gurobi [17]. DP denotes a dynamic-programming heuristic as a prevalent metaheuristic baseline.

Placement in different environments: In MEC, placement seeks satisfactory QoS for mobile users under wireless and resource constraints. Typical objectives include minimizing end-to-end delay, reducing migration cost, and improving availability. Representative examples include serverless scheduling that co-optimizes function placement and routing [48], delay-aware placement for autonomous-vehicle services [41], learning-based migration policies [42], coalition or MIS-based deployment and task assignment [43], [44], AoI-aware placement for digital twins [45], and multi-objective designs for vehicular or aerial edge networks [40], [46]. In SDN/NFV, VNF and service function chain (SFC) placement is a central topic, with objectives that balance latency, load, and availability. For instance, Li et al. [49] proposed JDBS to improve availability via joint deployment and backup in datacenters.

Despite overlapping objectives, these settings differ from SDP placement in large L3 networks. SDN/NFV studies often assume relatively static substrates and user distributions typical of datacenters. MEC focuses on stateful services and migration overhead. By contrast, SDP gateways are lightweight middleboxes that mediate user–service traffic directly on the path, so poor placement can create bottlenecks even when services themselves are fixed. Moreover, SDP placement must accommodate spatial–temporal mobility at scale without heavy migration. This leads to a problem that emphasizes minimizing user–service delay, achieving broad path coverage, and balancing gateway load under mobility.

Placement with methodological families: Placement studies in MEC and SDN/NFV largely fall into four methodological categories [60]. Table II compares the mainstream methods differences with ours. The first class uses mixed-integer linear programming (MILP) and convex formulations. For example, PlaceRAN [61] casts vNG-RAN function placement as a binary ILP that jointly respects split options and transport limits, achieving exact optimality on realistic topologies. In UAV systems, Sabzehali et al. [63] formulate the joint problem of UAV count, placement, and backhaul as an ILP and solve it with Gurobi, targeting coverage and backhaul feasibility. Promwongsa et al. [53] likewise start from an ILP for joint VNF placement/scheduling under latency deadlines and add scalable

heuristics for larger instances. Bari et al. [64] formalize VNF orchestration with an ILP and provide a dynamic-programming heuristic that attains within $\sim 1.3\times$ of optimal on real-topology traces.

The second class comprises heuristics and combinatorial optimization. For controller and VNF placement, Tohidi et al. [55] proved submodularity and applied a greedy scheme to balance delay and cost in distributed 5G vSDN control. Yu et al. [47] proposed BROAD to jointly optimize drone-mounted base-station placement and bandwidth allocation. For SFC embedding, Zheng et al. [52] used subgraph isomorphism with adaptive pruning to reduce complexity. Gao et al. [54] studied VNF placement in satellite edge clouds for large-scale IoT, using game-theoretic heuristics to minimize bandwidth cost and end-to-end delay. Qu et al. [65] minimized total UE energy in UAV-assisted MEC via alternating optimization over service placement, task offloading, compute allocation, and UAV trajectory.

A third class adopts evolutionary and metaheuristic methods. Alizadeh et al. [66] used a multi-objective genetic algorithm for SFC placement to explore the Pareto front. Rankothge et al. [67] analyzed GA-based algorithms for initial VNF placement and scaling under traffic changes. Bahrami et al. [68] proposed a binary hybrid NSGA-II method to approximate the Pareto set. Yang et al. [69] addressed segment-routing TE with header-overhead constraints, proving NP-hardness and proposing an improved ant-colony optimization scheme that balances load with limited control overhead.

The fourth class uses learning-based and Markov model approaches. Deep RL is employed for online decisions under dynamics: Pei et al. [50] designed a DRL algorithm for VNF placement to improve throughput, reduce latency, and balance load. Wang et al. [51] modeled service migration as a Markov decision process, providing a formal basis for cost-aware placement and migration. In the UAV context, Luo et al. [70] combined weighted k-means user clustering with Q-learning for content placement, illustrating hybrid heuristic-plus-learning designs. Wu et al. [71] proposed a method to predict traffic via multivariate time series and cast dynamic RRH–BBU mapping as an MDP, using an A3C policy to jointly optimize cost, energy, and load balancing in large-scale C-RANs.

However, these approaches face two practical limitations in the SDP setting. First, user mobility requires fast rebinding when a user moves. However, heavy optimizers or multi-epoch learners are often too slow for online operation, so the placement routine must remain lightweight. Second, learning-based methods can struggle to generalize when mobility patterns are unknown, highly stochastic, or shift across environments. In response, we propose a prediction-free, lightweight method that operates on observed per-epoch user locations and uses a PathHeatMap-guided, bi-objective greedy selection to balance delay and load variability.

III. Problem Formulation

We present the formal definition of the optimization problem for adaptive SDP placement. In the following, we first discuss the network model of this paper. Then, we formalize this problem as an Integer Linear Program (ILP) and prove that it is NP-hard.

TABLE III
Notation Table

Notation	Description
G	Network topology $G = (V, E)$.
$v \in V$	Network nodes $v \in V = \{v_1, v_2, \cdots\}$.
$e \in E$	Network link $e \in E = V \times V$.
$s_v \in S$	Network service $s_v \in S = \{s_1, s_2, \cdots\}$ at node v.
$p \in P$	Available SDPs $p \in P = \{p_1, p_2, \cdots\}$.
$u_v^t \in U$	User $u_v^t \in U = \{u_1, u_2, \cdots\}$ at node v at time t.
$f_{us}^t \in F$	Flow $f = (u, s) \in F$ from user u to service s at time t.
$t \in T$	Time series.
d_{ij}	Delay from node v_i to node v_j.
x_{vp}	1 if SDP p is placed at node v , 0 otherwise.
y_{fp}	1 if flow f is binding to SDP p, 0 otherwise.
$z_{fs} \in Z$	1 if user flow f can access service s, 0 otherwise.
b_f	Bandwidth requirement of user flow f.
b_p	Available bandwidth of proxy p.
λ	Weighting parameter in the objective function.
l_p	Load on proxy p.
N	Maximum number of proxies responsible for any service.
μ, σ	Mean and the standard deviation of proxy loads.
φ	Coefficient of variation.
ϕ	Normalized total delay.

System model: We focus on an IP (L3) network with a static topology during the evaluation window, representative of enterprise or campus networks where many users connect through access points (APs). Each AP is anchored to a nearby L3 router, and application-layer services (e.g., email, OA, authentication) run on fixed servers bound to specific L3 nodes. We further assume that SDP gateways proxy all *(user, service)* pairs to enforce Zero Trust. SDP gateways are deployable on router nodes. We assume the network deploys RADIUS servers to manage logins, which is popular in Zero Trust deployments. We use RADIUS association records to obtain, at each decision epoch, a snapshot of which users are attached to which APs, and thus to which L3 nodes.

For clarity, we adopt several common assumptions that do not restrict the approach: forwarding follows the routers' configured shortest-path policy. Server-side ECMP is disabled in our experiments (the framework can be extended if ECMP is enabled), and while demand varies over time, both the service locations and the L3 topology remain fixed.

Most importantly, users may move over time. However, we do not require any particular mobility model. The proposed algorithm does not learn or predict mobility. In evaluations, we use a Gaussian [9], [10] drift to generate diverse demand snapshots, and we also map the GeoLife [21], [22] real-world trace to the topology to demonstrate similar performance.

The notation used throughout the formulation is summarized in Table III. Specifically, the network topology is denoted as $G = (V, E)$, where $V = v_1, v_2, \ldots$ represents the set of nodes, and $E \in V \times V$ represents the edges. The network services are fixed at certain nodes, and we denote $s_v (\in S)$ as the service placed at node v_i. Similarly, u_v^t represents the user connected to node v at time t. The flow from user u to service s at time t is denoted as f_{us}^t. The network delay between nodes v_i and v_j is represented as d_{ij}. We introduce the decision variable x_{vp} to indicate whether an SDP p is placed at node v, and y_{fp} to denote whether flow f is bound to SDP p. Additionally, the bandwidth of flow f and SDP p are denoted as b_f and b_p, respectively.

Finally, we use λ as the weighting parameter in the objective function.

A. Constraints

With the network model and notation established, we now formalize the physical and operational constraints that must be satisfied in the SDP placement and reassignment process. These constraints can be divided into three categories: completeness constraints, resource constraints, and security constraints. We discuss each category in turn.

Completeness constraints: First, the decision variables in this problem are binary:

$$x_{vp}, y_{fp} \in \{0,1\}, \quad \forall v \in V, f \in F, p \in P, \tag{1}$$

where $x_{vp} = 1$ indicates that SDP p is placed at node v, and $x_{vp} = 0$ otherwise. Similarly, $y_{fp} = 1$ denotes that flow f is bound to SDP p, and $y_{fp} = 0$ otherwise. Second, each flow must be assigned to only one SDP, represented as:

$$\sum_{p \in P} y_{fp} = 1, \quad \forall f \in F. \tag{2}$$

Third, each SDP instance can be deployed at only one node. However, if the number of SDPs exceeds the network's requirements, some SDPs may remain undeployed. To account for this, we introduce the following constraint, where a value of 0 indicates that the SDP is not deployed, while a value of 1 means it is deployed at a node:

$$\sum_{v \in V} x_{vp} \leq 1, \quad \forall p \in P. \tag{3}$$

Similarly, each node can host at most one SDP instance. Although this constraint may be relaxed in cases where nodes have redundant resources, network middleboxes typically have strict resource limitations. Therefore, we assume the common scenario where each node supports only one SDP:

$$\sum_{p \in P} x_{vp} = 1, \quad \forall v \in V. \tag{4}$$

Resource constraints: For simplicity, this paper assumes that each flow requires the same amount of resources. Therefore, network resources are measured in terms of the number of flows each device can handle, rather than bandwidth. This assumption is represented as $b_f = 1$. In physical networks, the number of concurrent flows and bandwidth are interchangeable through simple calculations, ensuring that this assumption does not compromise generality. Furthermore, the assumption of uniform resource requirements per flow can be relaxed without affecting the designed method.

We model the SDP load constraint as follows:

$$\sum_{f \ in F} b_f y_{fp} \leq N, \quad \forall p \in P, \tag{5}$$

where N represents the maximum number of flows an SDP can handle.

Bandwidth constraints on individual links are beyond the scope of this paper. While bandwidth is important in networks, we do not impose specific constraints on it because SDP operates at the application layer, while independent network routing protocols manage lower-layer routing and bandwidth allocation. Since we do not modify or interfere with these mechanisms, we assume that routing protocols efficiently handle bandwidth allocation. Additionally, this paper does not consider constraints on the number of flows a service can process. We assume that each service has its own load-balancing mechanism co-located with backend servers.

Security constraints: In ZTN, security is usually promised through access control rules. While most placement algorithms [49], [50] do not consider access control rules as constraints, we incorporate them to reduce the search space of the proposed problem. Specifically, if a security policy prevents a flow from accessing a service, we exclude that flow from the assignment process. This reduces the number of decision variables, thereby improving computational efficiency.

We define the access control rule for a user u attempting to access a service s as:

$$z_{f_{us}} = \begin{cases} 1, & \text{if access is allowed,} \\ 0, & \text{if access is denied.} \end{cases}$$

Using this rule, we simplify the proposed problem with the following constraint:

$$z_f = 0 \Rightarrow \sum_{p \in P} y_{fp} = 0, \quad \forall p \in P. \tag{6}$$

B. Objective Function

Similar to other placement optimization studies [49], [50], the proposed objective is to optimize the QoS for users. The implementation of SDP gateways introduces additional delay overhead between users and services. Therefore, an ideal placement method should minimize these delays. One design objective is to reduce the connection delay between users and services, including application servers and the authentication controller. Secondly, if a single SDP gateway is overloaded with too many user-service bindings, it may become a bottleneck, leading to congestion and packet loss. Therefore, another design objective is to balance the load across the SDPs. In particular, we treat authentication as a service in the same formulation by modeling the controller as a service node. However, these two objectives can conflict. For instance, if the optimal SDP for a user exhausts its resources, we must allocate the user to another SDP, even though this may increase the connection delay compared to the optimal scenario. To balance these competing objectives, we introduce two weight factors that consider both delay and SDP load.

We define the proposed objective function as the minimization of the normalized total delay ϕ and the coefficient of variation (CV) φ [72], with a weighting parameter λ:

$$\min_{x,y} (1 - \lambda)\phi + \lambda\varphi, \tag{7}$$

where ϕ, φ, and λ are all within the range [0,1]. Below, we detail the computation of ϕ and φ.

Total delay minimization: We define the connection delay for a user flow f_{us} as the sum of the delay between the user and the

SDP (d_{up}) and the delay between the SDP and the service (d_{ps}). Our objective is to minimize the total delay across all user flows. The total network delay is given by:

$$Delay = \sum_{t\in T}\sum_{p\in P}\sum_{f^t_{us}\in F^t} y^t_{fp}(d_{up}+d_{ps}), \tag{8}$$

where the individual delay terms are determined using the SDP placement decision variable:

$$d_{up} = \sum_{t\in T}\sum_{v\in V}(x^t_{vp}d_{uv}),\ d_{ps} = \sum_{t\in T}\sum_{v\in V}(x^t_{vp}d_{vs}). \tag{9}$$

Substituting these expressions into the total delay function, we obtain:

$$Delay = \sum_{t\in T}\sum_{v\in V}\sum_{p\in P} x^t_{vp}\sum_{f^t_{us}\in F^t} y^t_{fp}(d_{uv}+d_{vs}). \tag{10}$$

Since we consider multiple objectives, we normalize the total delay $Delay^t$ into a value $\phi^t \in [0,1]$ using min-max normalization:

$$\phi = \frac{1}{\max\limits_{i,j\in V} d_{ij} - \min\limits_{i,j\in V} d_{ij}}(Delay - \min_{i,j\in V} d_{ij}). \tag{11}$$

Authentication as a service: In Zero Trust deployments, authentication traffic traverses user → SDP gateway → controller. In our formulation, the propagation component of authentication latency is captured by the same delay term $d_{up}+d_{ps}$, where we treat the controller as a service node s. Since the cryptographic processing time is constant for a given deployment, we omit it from the objective. Thus, incorporating authentication only requires enumerating these flows alongside other services, without changing the optimization structure.

Load balancing: To ensure balanced SDP utilization, we measure load balance using the CV [72]. CV is defined as: $CV = \sigma/\mu$, where σ is the standard deviation of the load distribution across SDPs, and μ is the mean load. CV is a dimensionless measure that quantifies the variability relative to the mean. A higher CV indicates a greater imbalance among SDP loads, whereas a lower CV suggests a more uniform distribution. In this paper, we minimize the CV as part of the proposed objective function to achieve a more balanced SDP load distribution. To calculate CV, we need to calculate the μ and σ. The mean SDP load at time t is computed as:

$$\mu^t = \frac{1}{|P|}\sum_{p\in P}\sum_{f\in F^t} y^t_{fp}, \tag{12}$$

where $|P|$ is the total number of SDPs, and the sum $\sum_{p\in P} y^t_{fp}$ gives the total number of flows assigned to SDP p. Similarly, the standard deviation of SDP loads is calculated as:

$$\sigma^t = \sqrt{\frac{\sum_{p\in P}\left(\sum_{f\in F^t} y_{fp} - \mu\right)^2}{|P|}}. \tag{13}$$

Using these values, we define the coefficient of variation φ as:

$$\varphi = \sum_{t\in T}\sigma^t/\mu^t. \tag{14}$$

Substituting ϕ and φ into (7), we obtain the proposed final objective function $\mathcal{P}_{\text{SDP}}$ as follows:

$$\mathcal{P}_{\text{SDP}}: \min_{x,y}\ (1-\lambda)\phi + \lambda\varphi,$$
$$\text{s.t.}\quad \text{Constraints (1)-(6).} \tag{15}$$

C. NP-Hardness Demonstration

In this section, we demonstrate that the SDP placement problem is NP-hard by reducing it to the well-known set cover problem [73], a classical NP-hard problem.

The Set Cover Problem (SCP) has already been proven to be an NP-hard problem [74] and is defined as follows:

Definition 1: $\mathcal{P}_{\text{SCP}}$: Given a universe set U of elements, and a collection of sets $C_1, C_2, \ldots, C_m$ where each C_i is a subset of U, the objective is to select a minimum number of sets from $\{C_1, C_2, \ldots, C_m\}$ such that every element in U is contained in at least one selected set.

Now we have the following theorem:

Theorem 1: The $\mathcal{P}_{\text{SDP}}$ in Eq. 15 is an NP-hard problem.

Proof: To establish the reduction, we map the components from $\mathcal{P}_{\text{SCP}}$ to $\mathcal{P}_{\text{SDP}}$. Specifically, each element $e_j \in U$ corresponds to a service $s_j \in S$ that must be covered by SDPs. Each set C_i represents an SDP instance p_i, which can serve multiple services. If $e_j \in C_i$, then SDP p_i handles service s_j. Then, the objective of the set cover problem is to minimize the number of sets selected. Similarly, in the $\mathcal{P}_{\text{SDP}}$, we aim to minimize the total connection delay, which is closely related to reducing the number of SDPs while ensuring all services are covered. Specifically, deploying an SDP increases the connection delay. Therefore, minimizing the total delay inherently requires minimizing the number of SDPs, while the proposed constraints ensure that every flow is covered.

Since the $\mathcal{P}_{\text{SCP}}$ is NP-hard, and we have established a polynomial-time reduction, it follows that $\mathcal{P}_{\text{SDP}}$ is also NP-hard. ■

IV. Method Design

Overview: To solve $\mathcal{P}_{\text{SDP}}$ mentioned in Section III-C, we propose a two-step SDP placement architecture with offline deployment and online reassignment. In the offline stage, we introduce a flow aggregation mechanism called *PathHeatMap-guided SDP embedding*, which constructs a heatmap of user-server path overlaps to guide the placement of SDPs at high-impact nodes. This mechanism prioritizes direct flow capture by identifying convergence hotspots in the network topology. For the flows not covered by on-path SDPs, the system performs a constrained fallback assignment to minimize detour overhead. In the online stage, we design a lightweight reassignment method, *deferred SDP rebinding*, which reacts to user mobility by temporarily assigning displaced users to nearby SDPs and triggering global rebinding only when necessary. This approach enables a fast response to movement without frequent global recomputation.

Specifically, as shown in Fig. 1, the placement controller has the following three key modules: (1) the PathHeatmap-guided

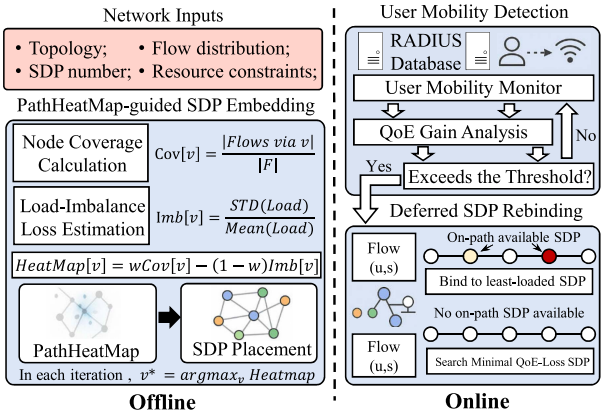


Fig. 1. Overview of the proposed architecture, which consists of an offline SDP embedding module and two online modules for mobility detection and deferred flow reassignment.

SDP Embedding Module, (2) the User Mobility Detection Module, and (3) the Deferred SDP Rebinding Module. Next, we illustrate how the architecture works with an example.

Initially, we start with a traditional network $G = (V, E)$ and aim to deploy a fixed number of SDPs across a network. The PathHeatmap-guided SDP embedding module inputs historical network information—such as the network topology, available bandwidth, service locations, and initial user distributions. This module runs an offline algorithm (Algorithm 1) to determine the optimal initial SDP placement and bind the user flows to each SDP.

Next, the User Mobility Detection Module establishes a flow information table, which includes data on propagation delays, user access points, and the SDPs to which each flow is bound. This module continuously monitors user movement by tracking changes in propagation delay over time. If significant changes in delay are detected, the module updates the user's connected access point information accordingly.

Finally, the Dynamic Re-assignment Module handles user reassignment based on the updated mobility data. If a user is connected to an SDP and an optimal alternative SDP is available, the reassignment module rebinds the user to this new SDP to reduce connection overhead. If no suitable SDP is available, the flow information is stored in a rebind queue. When the number of buffered flows exceeds a predefined threshold, the module recalculates optimal SDP placements and establishes new instances for the buffered flows.

We now describe each of the three modules in detail.

A. PathHeatmap-Guided SDP Embedding

This module places SDP gateways on the L3 topology. The key idea is to position SDP gateways on router nodes that intercept many user–service paths so that most flows are served without detour, thereby reducing delay while avoiding load hotspots. To this end, we summarize path demand with a `PathHeatMap` and select gateway locations using a lightweight bi-objective criterion that trades off path coverage and load balance.

Given the routing table and the set of allowed user–service pairs, we proceed as follows. For each pair (u, s), we compute its route node set $\text{Route}(u, s)$ and the number of active users at u that are permitted to access s, denoted n_{us}. The *coverage* contributed by a node v is the total demand (by user counts) whose shortest path includes v:

$$\texttt{Coverage}[v] = \{(u, s) \mid v \in \text{GetPath}(u, s)\}, \quad (16)$$

where `Coverage` records the set of flows traversing each node in a network. Specifically, if $\texttt{Coverage}[v_1] = \{f_1, f_2, f_3\}$, it indicates that flows f_1, f_2, and f_3 include node v_1 in their respective routing paths. Since individual flows may traverse multiple nodes, a given flow may appear in `Coverage` entries of several nodes. The size of each entry reflects the number of flows that pass through the corresponding node, serving as a measure of its traffic centrality.

We restrict candidates to the union of these paths to prune irrelevant nodes. However, placing gateways solely by `Coverage` can create bottlenecks if a high-coverage node attracts too many bindings. We therefore define a load-imbalance penalty, denoted $\texttt{imbalance}(v)$. For a candidate node v, we temporarily assign each covered flow to the least-loaded gateway on its shortest path and obtain the resulting per-gateway loads. We then take the coefficient of variation of these loads as $\texttt{imbalance}(v)$.

Finally, we combine the two factors into a bi-objective heat score:

$$\text{PathHeatMap}[v] = w \cdot \frac{|\texttt{coverage}[v]|}{|F|} - (1 - w) \cdot \texttt{CV}, \quad (17)$$

where $w \in [0, 1]$ is the trade-off weight. Here $|\texttt{coverage}[v]|/|F|$ denotes the normalized coverage of v, and $\text{CV}(v) = \sigma/\mu$ is the coefficient of variation of provisional per-gateway loads when evaluating v. To estimate CV, we temporarily attach each covered flow to the least-loaded on-path proxy in $P \cup \{v\}$ and compute the resulting load variance and mean. Because the imbalance term is subtracted, candidates that induce larger load skew receive lower scores and are less likely to be selected. Placement then proceeds in k iterations: at each step, we add the candidate that maximizes the marginal gain $\Delta(v \mid P) = \text{PathHeatMap}(P \cup \{v\}) - \text{PathHeatMap}(P)$, where P is the set of already selected gateways.

Algorithm explanation: Algorithm 1 performs greedy SDP placement using a coverage–balance score. It takes as input the topology $G{=}(V, E)$, the number of SDPs k, users U, the access list Z, and services S. The algorithm first builds the flow set F by filtering $(u, s) \in U{\times}S$ with $z_{us} = 1$. For each flow $f \in F$, it calls $\texttt{GetPath}(f)$ to obtain the routed node sequence and records that f contributes to $\texttt{Coverage}[v]$. This yields $\texttt{Coverage}[v]$ as the set of flows whose routing path includes v.

The greedy placement then begins (lines 10–26). At each iteration, every candidate node $v \in V \setminus P$ is evaluated by two terms. First, the normalized coverage score is computed (line 12), which rewards nodes that lie on many flow paths. Second, the imbalance penalty is computed (lines 13–19): a temporary proxy set $P \cup \{v\}$ is formed and per-proxy loads are initialized to zero (line 13). For each $f \in F$, we derive the candidate

Algorithm 1: Proxy Placement Algorithm.

Input : Topology $G = (V, E)$, Number of SDP k, Users U, Access list Z, Services S, Flows $F = \emptyset$.
Output: SDP set P.

1 **foreach** $v \in V$ **do** PathHeatMap[v] $\leftarrow 0$;
2 **foreach** $(u, s) \in U \times S$ **do**
3 **if** $z_{us} == 1$ **then** $F \leftarrow F \cup \{f_{us}\}$;
4 **end**
5 **foreach** $f \in F$ **do**
6 **if** $v \in$ GetPath(f) **then**
7 Coverage[v] $\leftarrow$ Coverage[v] $\cup$ f
8 **end**
9 **end**
// Greedy SDP Placement.
10 **for** $i = 1$ **to** k **do**
11 **for** $v \in V \setminus P$ **do**
12 Score[v] $\leftarrow w \cdot |Coverage[v]|/|F|$
13 **foreach** $p \in P$ **do** $Load_{tmp}[p] \leftarrow 0$;
14 **for** $f \in F$ **do**
15 $C_{tmp} \leftarrow (P \cup \{v\}) \cap$ GetPath(f)
16 **if** $C \neq \emptyset$ **then**
17 $p_{\min} \leftarrow \arg\min_{p \in C}$ load[p]
18 load[$p_{\min}$] $\leftarrow$ load[$p_{\min}$] $+ 1$
19 **end**
20 **end**
21 **if** $\sum_p$ load[p] $\neq 0$ **then**
22 PathHeatMap[v] $\leftarrow$ Score[v] - $(1 - w) \cdot CV(Load_{tmp})$
23 **end**
24 **end**
25 $v^\star \leftarrow \arg\max_{v \in V \setminus P} PathHeatMap[v]$
 $P \leftarrow P \cup \{v^\star\}$
26 **end**
27 **return** P.

on-path proxy set (line 15). If this set is nonempty, the flow is temporarily assigned to the least-loaded SDP on its path to estimate loads (lines 16–18). From these provisional loads, the coefficient of variation is computed, $\text{CV}(\text{Load}_{\text{tmp}}) = \text{std}/\text{mean}$. The node's PathHeatMap then combines coverage and imbalance with the specified weight (line 21). The node with the largest PathHeatMap value over $V \setminus P$ is selected as an SDP and added to P, and the loop proceeds to the next iteration until k gateways have been chosen.

Theorem 2: The worst time complexity of the Algorithm 1 is $O(|P||U||S||V|)$.

Proof: In the initialization, setting the decision variable x_{vp} for all nodes v and SDPs p requires $O(|V||P|)$ operations. Next, initializing the PathHeatMap for all nodes takes $O(|V|)$ steps. Then, constructing the flow set F involves forming $O(|U||S|)$ user-service pairs, with filtering based on Z, leading to a worst-case complexity of $O(|U||S|)$. In the PathHeatMap initialization, the function GetPath(f) calls the underlying routing function to compute flow paths. In most scenarios, the proposed algorithm runs after the network has already been established. Thus, instead of recomputing routing paths using Dijkstra's algorithm ($O(|E| + |V| \log |V|)$), we can query the lower-layer API, which operates in constant time $O(1)$. Given that $|F| \leq |U||S|$, this results in $O(|U||S|)$. In the SDP placement part, each iteration selects the node v_{tmp} with the highest flow volume, requiring $O(|V|)$ operations. Next, the total number of iterations is bounded by $|P|$, as at most $|P|$ SDPs are placed. Then, the update step removes all flows passing through v_{tmp} and recalculates GetPath(f) for the remaining flows. In the worst case, all flows are checked once per iteration, each flow touches at most $O(|V|)$ nodes, leading to a complexity of $O(|P||U||S||V|)$. Therefore, the total complexity is dominated by the placement phase, yielding an overall complexity of $O(|P||U||S||V|)$. ■

B. User Mobility Detection Module

User mobility may invalidate a previously good SDP binding: when a user moves, the old gateway may introduce detours and concentrate load, degrading QoE. Traditional learning-based approaches may be brittle across environments and difficult to maintain at scale. We therefore seek a prediction-free, low-overhead way to observe the user location.

In Zero Trust deployments, RADIUS is a prevalent control-plane service for accounting. The RADIUS association event provides a lightweight record linking a user identifier to an access point and time, yielding an up-to-date snapshot of the user–node distribution without extra instrumentation. By leveraging these ubiquitous RADIUS records, the module monitors mobility with minimal operational cost and without assumptions about mobility patterns, enabling timely and portable SDP rebinding across diverse networks.

The module maintains two tables throughout its operation. One table contains the last updated data sent to the deferred SDP rebinding module, while the other records real-time updates that have not yet been transmitted. This mechanism prevents excessive and rapid reassignments. When the number of moved users exceeds a predefined threshold, an updated table is sent to the deferred SDP rebinding module, triggering a partial reassignment process.

Tracking user's mobility: To track user mobility and update the table, several methods can be used in networks. The Simple Network Management Protocol (SNMP) and the Network Configuration Protocol (NETCONF) allow administrators to query Access Controllers (ACs) for user session details, including the associated AP. These protocols are widely adopted in networks for monitoring and performance evaluation. Another method is leveraging the Remote Authentication Dial-In User Service (RADIUS) [75], which is commonly integrated into authentication systems. Administrators can query the RADIUS database to retrieve users' MAC addresses and their associated APs.

In this paper, we use RADIUS accounting for mobility tracking. We achieve this by connecting to the RADIUS database using `pymysql` [76] and querying the `radacct` table. The `nasipaddress` field provides the AP's IP address, while the `callingstationid` field contains the user's unique identifier, typically the MAC address. Moreover, we add a query

Algorithm 2: Assignment Algorithm.

Input : Topology $G = (V, E)$, Deployment matrix x_{vp}, Flow distribution F.
Output: Decision Variables $y_{fp}, \forall f \in F, \forall p \in P$.

1 **foreach** $f \in F, p \in P$ **do** $y_{fp} \leftarrow$ None;
2 **foreach** $p \in P$ **do** Load[p] $\leftarrow 0$;
3 **foreach** $f \in F$ **do**
4 $C_{tmp} \leftarrow \emptyset$ // Candidate SDPs
5 PathList $\leftarrow$ GetPath(f)
6 **foreach** $v \in$ PathList **do**
7 **if** $x_{vp} = 1$ ***and*** Load[p] $< N$ **then**
8 $C_{tmp} \leftarrow C_{tmp} \cup \{p\}$
9 **end**
10 **end**
11 $p_{tmp} \leftarrow \arg\min_{p \in C_{tmp}}$ Load[p]
12 **if** $p_{tmp} \neq$ *None* **then**
13 Load[p_{tmp}] $\leftarrow$ Load[p_{tmp}] $+ 1$
14 $F \leftarrow F \setminus \{f\}$; $y_{fp_{tmp}} \leftarrow 1$
15 **end**
16 **end**
17 **foreach** $f_{us} \in F$ **do**
18 $O_{tmp} \leftarrow \emptyset$ // Candidate SDPs
19 **foreach** $p \in P$ **do**
20 **if** Load(p) $< N$ **then**
21 $o1_p \leftarrow$ Normalize($d_{up} + d_{ps}$)
22 $o2_p \leftarrow$ CV_increment(p)
23 $O_{tmp} \leftarrow O_{tmp} \cup \{p\}$
24 **end**
25 $p_{tmp} \leftarrow \arg\min_{p \in O_{tmp}} ((1-\lambda)o1_p + \lambda o2_p)$
26 $y_{fp_{tmp}} \leftarrow 1$
27 **end**
28 **end**

condition where the acctstoptime field is Null, which means the session is active.

Threshold for reassignment: We periodically query the API and compare the retrieved data with the last updated table to identify users on the move. Since mobility affects SDP assignments, a suboptimal assignment can increase connection delay. To determine if reassignment is necessary, we estimate the increased delay.

Given that the network's propagation delay between nodes is known if a user connected to SDP p moves from node u to u', we compute the delay change as $d_{u'p} - d_{up}$. If this value is negative, the user has moved closer to the SDP, and reassignment is unnecessary. Otherwise, we normalize the delay increase using min-max normalization:

$$\theta_{f \to f'} = \frac{d_{u'p} - d_{up} - \min\limits_{i,j \in V} d_{ij}}{\max\limits_{i,j \in V} d_{ij} - \min\limits_{i,j \in V} d_{ij}}. \tag{18}$$

At each time interval, we compute the total increased delay by summing all positive θ values:

$$\Theta = \sum_{f,f'} \max\{\theta_{f \to f'}, 0\}. \tag{19}$$

We define a threshold based on the objective value φ to determine when reassignment is necessary. The threshold is set as 0.05φ. If $\Theta < 0.05\varphi$, the system continues updating user distributions without triggering reassignment. If $\Theta \geq 0.05\varphi$, the updated user distribution is sent to the reassignment module, initiating a recalculation of optimal SDP assignments.

This threshold allows for a 5% tolerance before triggering reassignment. Once the increase in connection delay exceeds this margin, the system dynamically optimizes SDP assignments to maintain efficient network performance.

C. Deferred SDP Rebinding Module

This module manages the assignment and reassignment of flows to SDPs. As outlined in the Section III, its objective is to solve $y_{fp}, \forall f \in F, p \in P$. Additionally, it ensures load balancing across SDPs while satisfying resource constraints. To achieve this, the module takes SDP placements from the PathHeatmap-guided SDP embedding module, user distribution from the mobility tracking module, and computes the binding of each flow to an SDP using Algorithm 2. When the system detects a significant change in user distribution, *i.e.,* when the delay objective exceeds the threshold, the module dynamically reassigns SDPs to flows.

The algorithm follows these steps. First, it determines the routing path of each flow by calling GetPath(f). If an SDP is available on the flow's path and satisfies resource constraints, the module assigns the least-loaded SDP to the flow. This method minimizes connection delay since the flow does not need to take detours while also balancing SDP loads by selecting the least congested option.

If no SDP is available on the flow's path, the module temporarily buffers the flow and processes it later. This approach prioritizes flows with on-path SDPs to prevent suboptimal flows from prematurely consuming SDP resources, which could lead to a double-loss scenario where neither load balancing nor optimal delay is achieved. Once all feasible flows are assigned, the module processes the remaining suboptimal flows. Since the SDP placement module ensures that most flows have an on-path SDP, the number of remaining suboptimal flows is expected to be minimal.

For each suboptimal flow, the module evaluates two objectives: the delay objective value φ for selecting each available SDP and the load balancing objective ϕ. These values are combined as $(1-\lambda)\varphi + \lambda\phi$. The SDP with the lowest computed value is selected for each flow, ensuring that both delay minimization and load balancing objectives are met.

We present the flow assignment algorithm in Algorithm 2. In lines 1-3, we initialize the necessary variables. The function Load(p) computes the current load of SDP p, which is initially set to zero. The assignment process begins in lines 3-14, where the algorithm handles flows that have an optimal on-path SDP. In line 4, we create a temporary set C_{tmp} to store candidate SDPs for flow f. Lines 5-10 compute the flow's routing path and add any SDPs found along the path to C_{tmp}. The algorithm then selects the on-path SDP with the minimal load as the assignment for flow f.

TABLE IV
SIMULATION TOPOLOGIES ATTRIBUTES

Name	nodes	links	Services	SDPs	Users
Bics	33	48	10-30	10-30	200
Chinanet	42	66	10-30	10-30	200
VtlWavenet	92	96	10-80	10-80	1,000
UsCarrier	158	189	10-80	10-80	1,000
Cogentco	197	243	10-80	10-80	1,000

If no on-path SDP is available, *i.e.,* $p_{tmp} == None$, the flow is processed in lines 17-28. In this phase, the algorithm evaluates all available SDPs, computing both the delay and load-balancing objectives for each possible assignment. The flow is then assigned to the SDP with the minimal combined objective value. This step performs an exhaustive search over possible assignments for a small subset of flows. Since the majority of flows are assigned in the previous step (lines 3–16), only a few flows require processing in this phase.

Theorem 3: The worst time complexity of Algorithm 2 is $O(|F||V|)$.

Proof: The complexity analysis of the assignment algorithm consists of two main phases: on-path SDP assignment and secondary SDP optimization. In the first phase, for each flow $f \in F$, the algorithm retrieves its shortest path using $\texttt{GetPath}(f)$, which operates in $O(1)$ time assuming precomputed paths. It then iterates over the nodes along the path, checking for candidate SDPs, which takes at most $O(|V|)$ in the worst case. Once candidate SDPs are identified, the algorithm selects the least-loaded SDP using an $O(|P|)$ min-operation, assigns the flow, and updates the load tracking in $O(1)$. Since this process runs for every flow, the total complexity of the first phase is $O(F(|V| + |P|))$. In the second phase, any remaining unassigned flows undergo a secondary optimization step, where each flow iterates over all SDPs to check load constraints and compute optimization metrics, both of which take $O(1)$ operations per SDP. The best SDP is then selected using another $O(|P|)$ min-operation, leading to a total complexity of $O(|F||P|)$ for this phase. Combining both phases, the overall complexity of the algorithm is $O(|F||V|)$, making it practical for online execution. ■

V. EVALUATION

We evaluate the proposed system using large-scale simulations on an Ubuntu 22.04 server with a 98-core Intel Xeon CPU and 200 GiB RAM. Link delays are synthesized from node geolocations via a standard linear regression model, following prior work [77], [78]. Services are fixed at selected nodes, and we vary the number of users and SDP gateways across experiments.

Topologies: We use five real L3 topologies from the Internet Topology Zoo [20]: Bics (33 nodes), Chinanet (45 nodes), VtlWavenet (87 nodes), USCarrier (165 nodes), and Cogentco (197 nodes). Topology details appear in Table IV. To enable fair comparisons with exact and evolutionary baselines whose runtimes grow rapidly, we also include small instances (e.g., Abilene, 11 nodes) in scalability experiments.

Demand and mobility: Users attach to APs anchored to L3 nodes, and flows are defined by allowed (user, service) pairs. For synthetic studies, we generate time-varying demand via a Gaussian drift to produce diverse snapshots. To validate with real mobility, we map the GeoLife [21], [22] dataset (182 users) onto the Chinanet topology, using GPS displacements to trigger node changes. In both cases, the controller consumes a per-epoch snapshot of user locations (as would be obtained from RADIUS association records) and does not learn or predict a mobility model.

Baselines: We compare against two empirical methods, Random and Gateway (nearest assignment), and four stronger baselines: a Dynamic Programming heuristic, a Convex relaxation [18], a MILP formulation [17], and NSGA-II for bi-objective exploration. These methods are widely used for placement problems. NSGA-II is configured to trace empirical Pareto fronts (e.g., 100 generations, 100 offspring per generation). For heavy solvers, experiments are run on tractable topologies within the 24-hour time budget. If the time budget is exceeded, we record a timeout.

Experiments and metrics: We conduct a comprehensive evaluation. First, we examine the delay–CV trade-off via Pareto analysis (using NSGA-II as a reference envelope). Second, we sweep the number of SDPs and report QoE (lower is better). Third, we measure the scalability of wall-clock runtime as the topology size and user count grow. Fourth, we include an authentication-latency extension that treats the controller as a service endpoint within our objective. Fifth, we perform a threshold ablation that varies the rebinding trigger (0.01–0.08) to quantify the quality–overhead knee. Finally, we present a case study with the GeoLife [21], [22] dataset.

A. Pareto Analysis

We perform a Pareto analysis against the advanced baselines. For NSGA-II, we run 100 generations with 100 offspring per generation, then plot the Pareto front and a subsample of nearby points. For the MILP formulation, the convex relaxation, the Dynamic Programming heuristic, and our method, we sweep the scalarization weight $w \in [0, 1]$ to assess sensitivity. This experiment characterizes both performance and robustness to the choice of w.

Performance: In Fig. 2, each cluster corresponds to a method obtained by sweeping the weight $w \in [0, 1]$. Orange crosses denote NSGA-II Pareto-front points, gray dots are other NSGA-II samples (subsampled), red markers are our method, blue squares are Dynamic Programming, purple markers are MILP, and green markers are the convex relaxation. On both Bics and Chinanet, our red cluster concentrates near the low-delay and low-CV corner, overlapping the empirical Pareto envelope traced by NSGA-II. DP and MILP lie nearby but typically at slightly higher delay and CV, whereas the convex relaxation drifts toward the upper-right. The insets highlight that the best observed operating points are located around the red cluster, indicating that our method attains near-front performance.

Sensitivity: For our method, DP, MILP, and the convex relaxation, the points collapse into tight clusters on both topologies,

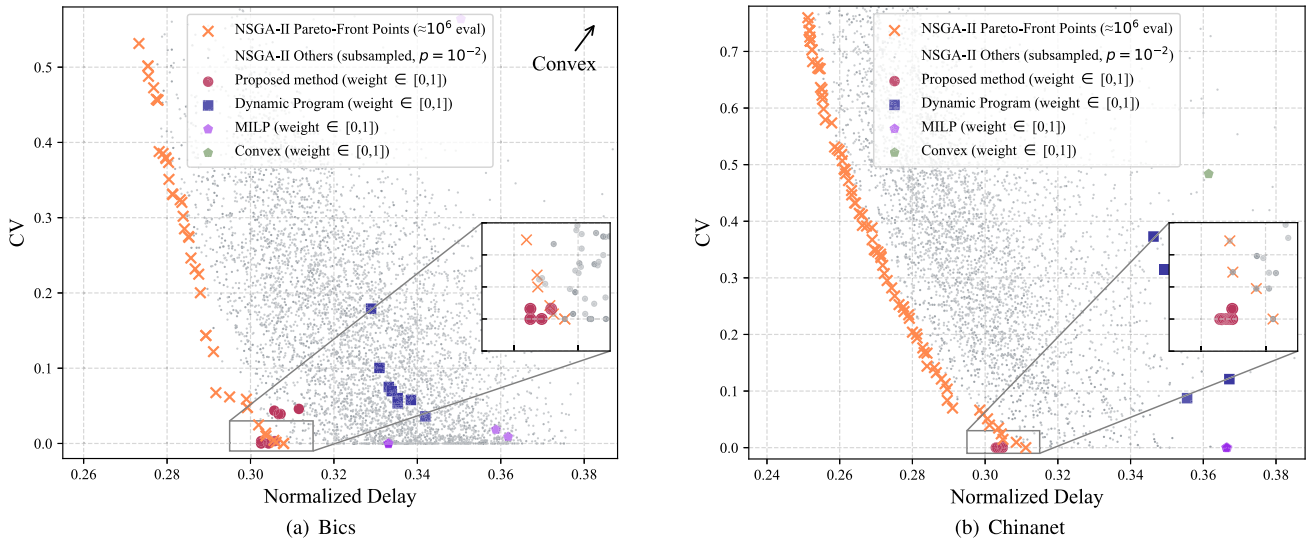


Fig. 2. Pareto trade-off between normalized delay and CV on (a) Bics and (b) Chinanet. Lower values are better. Insets zoom into the knee region, where our method forms a compact cluster at the low-delay, low-CV corner.

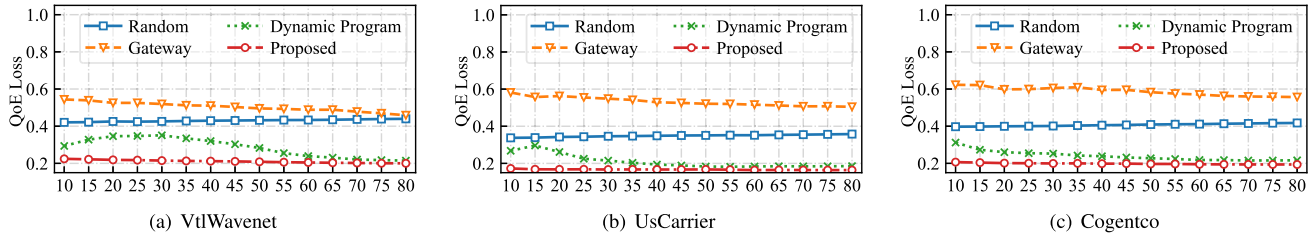


Fig. 3. QoE performance versus the number of proxies under static user distribution across three different topologies: VtlWavenet, UsCarrier, and Cogentco.

indicating minimal sensitivity to w. Varying w does not materially change the delay–variability trade-off achieved by these methods. By contrast, NSGA-II spans a continuous envelope, as expected for a population-based multi-objective optimizer.

B. Parameter Analysis

In this experiment, we evaluate the performance of the proposed SDP placement modules under a static user distribution, comparing them with baseline approaches. We conduct comprehensive experiments by varying the number of SDPs and services to observe their impact on the design objective.

Small-scale networks: We compare the proposed method with MILP, a convex relaxation, Dynamic Programming (DP), Gateway (nearest assignment), and Random on two small topologies while varying the proxy count from 10 to 30. For each configuration, we report the average QoE, where smaller values indicate better performance. Within each proxy group, bars follow the legend order: Proposed (blue, diagonal hatch), DP (orange), Random (green), Gateway (teal), Convex (purple), and MILP (pink, grid hatch).

Results: According to Fig. 4, the proposed method attains the lowest QoE in 9 of 10 settings. Quantitatively, in the Bics topology, the proposed method's best value at 10, 15, 25, and 30

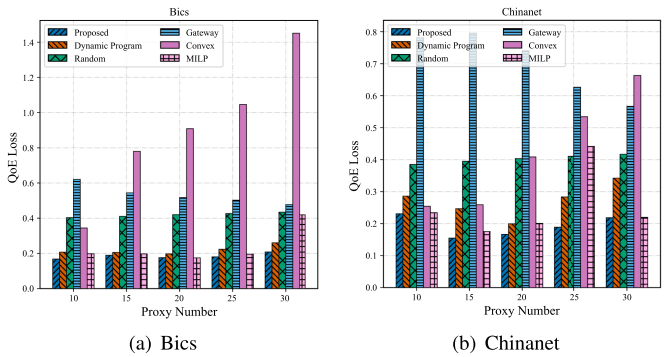


Fig. 4. QoE versus proxy count on (a) Bics and (b) Chinanet.

proxies with QoE 0.168, 0.190, 0.180, and 0.208, respectively. Relative to the strongest baseline at each setting, these are lower by 15.2% (*vs.* MILP at 10 proxies), 3.7% (*vs.* MILP at 15 proxies), 7.8% (*vs.* MILP at 25 proxies), and 19.9% (*vs.* DP at 30 proxies). At 20 proxies, MILP edges out our Proposed method by 0.0017 ($\approx$0.95%). Convex degrades markedly as proxies increase (from 0.345 to 1.451), Gateway improves modestly but remains high (e.g., $0.621 \rightarrow 0.478$), and Random stays around $0.40 \sim 0.43$. In Chinanet topology, the proposed method is best for all proxy counts with QoE 0.231, 0.155, 0.167, 0.189, and

TABLE V
SCALABILITY ANALYSIS

Network size	Network Topology	Flows	Proposed	Random	Gateway	Dynamic Program	Convex	MILP	NSGA-II (100-gen)
11 nodes	Abilene	100	0.01s	0.002s	0.002s	1.06s	0.96s	0.38s	451.74s
33 nodes	Bics	2000	1.30s	0.12s	0.08s	6.48s	4773.31s	2900.41s	2897.48s
45 nodes	Chinanet	2000	2.28s	0.18s	0.13s	8.48s	8573.55s	8622.47s	7,684.08s
87 nodes	VtlWavenet	4000	3.67s	3.86s	2.28s	841.96s	-	-	-
165 nodes	UsCarrier	4000	7.94s	7.57s	4.57s	993.46s	-	-	-
197 nodes	Cognetco	4000	12.16s	9.94s	6.16s	1149.36s	-	-	-

Note: "–" indicates runtime $>$ 24 hours.

0.219 at 10–30 proxies. The margins over the strongest baseline are 1.45% (*vs.* MILP at 10), 11.9% (*vs.* MILP at 15), 16.4% (*vs.* DP at 20), 33.3% (*vs.* DP at 25), and 0.54% (*vs.* MILP at 30). Convex again worsens with more proxies ($0.254 \rightarrow 0.663$), Gateway declines but remains substantially higher ($0.781 \rightarrow 0.567$), and Random slowly drifts upward ($0.385 \rightarrow 0.417$). Overall, Proposed reduces QoE by about 11% on average relative to the best competing method across all settings (median $\approx 9.8\%$). The advantage is consistent across proxy densities and topologies.

Large-scale networks: We also evaluate our method on larger topologies. Because the heavy solvers (MILP, Convex, NSGA-II) frequently exceed a 24-hour limit on these graphs (Section V-C), they are impractical in these cases. We therefore compare against Dynamic Program, Gateway (nearest assignment), and Random while sweeping the number of proxies from 15 to 80. For each configuration we report the average QoE (smaller is better). In Fig. 3, lines follow the legend order: Proposed (red), Dynamic Program (green), Random (blue), and Gateway (orange).

Results: According to Fig. 3, the proposed curve is lowest for every proxy count from 15 to 80. The advantage is sizable at small and moderate proxy densities and remains measurable even at high densities. For example, on USCarrier with 15 proxies, the proposed method achieves QoE 0.168 versus 0.296 for Dynamic Program, a 43% reduction. On VtlWavenet at 30 proxies, Proposed is 0.215 versus Dynamic Program's 0.351 (38% lower), and on Cogentco at 80 proxies, Proposed is 0.194 versus Dynamic Program's 0.216 ($\approx 10.5\%$ lower). Averaged over all proxy counts and the three topologies, Proposed is about 19% lower than the strongest competing baseline on each setting (median improvement $\approx 14\%$). Dynamic Program and Gateway improve gradually as proxies increase, Random drifts slightly upward, and Proposed remains near-flat at a low level (e.g., $\sim 0.22 \rightarrow 0.20$ on VtlWavenet, $0.168 \rightarrow 0.164$ on USCarrier, and $0.204 \rightarrow 0.194$ on Cogentco). These results prove that the proposed placement maintains favorable QoE across ISP-scale topologies while heavy solvers are computationally infeasible.

C. Scalability Analysis

We evaluate runtime across six topologies that range from 11 to 197 nodes and from 100 to 4000 flows. The proposed method remains fast on all instances and, empirically, grows near linearly with topology size (from 0.01 s at 11 nodes to 12.16 s at 197 nodes). Exact solvers and multi-objective search scale poorly, frequently exceeding the 24-hour cap on medium and large networks.

Results: According to Table V, the proposed method increases smoothly with network size, remaining under 13 s even for 197 nodes and 4000 flows. From 11 to 197 nodes, runtime grows from 0.01 s to 12.16 s. Moreover, the proposed method demonstrates large advantages over heavy solvers. On small and medium topologies, the proposed method is orders of magnitude faster than exact and evolutionary baselines. On Abilene (11 nodes), it is $4.5 \times 10^4\times$ faster than NSGA-II, $96\times$ faster than the convex relaxation, and $38\times$ faster than MILP. On Bics (33 nodes), the gaps remain large: $2.23 \times 10^3\times$ over NSGA-II, $3.67 \times 10^3\times$ over Convex, and $2.23 \times 10^3\times$ over MILP. On Chinanet (45 nodes), speedups are $3.37 \times 10^3\times$ (NSGA-II), $3.76 \times 10^3\times$ (Convex), and $3.78 \times 10^3\times$ (MILP). For larger networks (87–197 nodes), Convex, MILP, and NSGA-II all exceed 24 hours, whereas the proposed method finishes in 3.67–12.16 s. Dynamic Programming remains feasible but is much slower, requiring $106\times$ (11 nodes), $5.0\times$ (33 nodes), $3.7\times$ (45 nodes), and 95–$229\times$ (87–197 nodes) the runtime of the proposed method.

Random and Gateway are very fast due to their simplicity. Even so, the proposed method is within a small constant factor: on 165–197 nodes it is within $1.05\times$–$1.22\times$ of Random and within $1.74\times$–$1.97\times$ of Gateway. Coupled with the QoE results, these runtimes show that the proposed approach offers near-heuristic speed with markedly better solution quality.

TABLE VI
MEAN AUTHENTICATION DELAY (S)

Method / Topo	Bics	Chinanet	VtlWavenet	USCarrier	Cogentco
Baseline	2.3414	2.7762	2.7579	2.7144	4.1989
Proposed	2.2971	2.4609	2.6808	2.5389	3.8893
Reduction (%)↓	1.89%	11.36%	2.80%	6.47%	7.37%

Baseline: deployment without the proposed framework.

D. Authentication Delay

Table VI shows consistent reductions across all five topologies. The largest improvement appears on Chinanet (11.36%), followed by Cogentco (7.37%) and USCarrier (6.47%). VtlWavenet and Bics also benefit, with smaller yet measurable gains (2.80% and 1.89%).

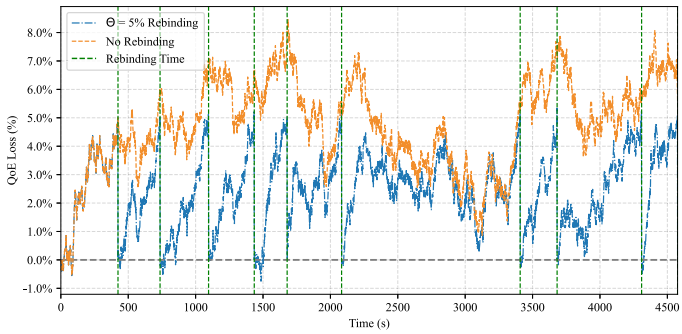


Fig. 5. QoE loss over time and the rebinding threshold on Chinanet. Lower is better.

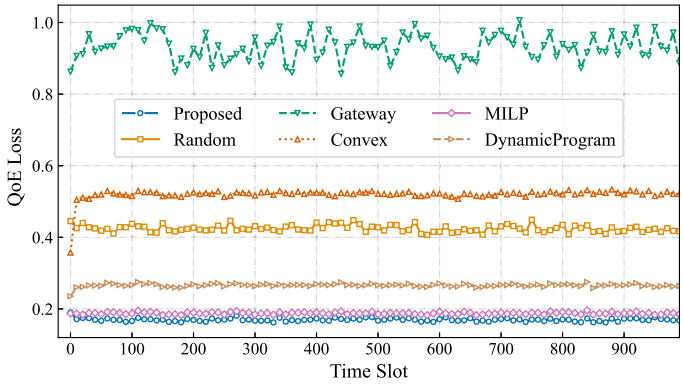


Fig. 6. Performance under the GeoLife [21], [22] dataset and the Chinanet topology.

TABLE VII
REBINDING TRIGGERS WITHIN 1 HOUR UNDER DIFFERENT THRESHOLDS

θ	Rebinding Count	Avg. QoE	θ	Rebinding Count	Avg. QoE
0.01	81	0.190757	0.05	5	0.197497
0.02	40	0.191727	0.06	1	0.201772
0.03	19	0.193847	0.07	1	0.201581
0.04	14	0.194910	0.08	1	0.200699

Excluding Chinanet, the reduction percentage tends to increase with topology size. This trend is expected: larger networks exhibit longer average propagation paths, so optimizing placement and assignment yields proportionally greater savings. Chinanet stands out because it has a hub-and-spoke, star-like structure, where a central node connects many others. As a result, under the baseline assignment, many users are topologically distant from gateways/controllers, and our optimization shortens those routes substantially, producing the largest observed reduction.

These results indicate that minimizing propagation distance lowers end-to-end authentication latency while the cryptographic processing time remains unchanged. Although the magnitude varies with topology scale and path diversity, the direction is uniform: our method does not increase authentication latency in these experiments. Overall, the evidence supports that our framework improves the security–performance trade-off by reducing authentication overhead.

E. Rebinding Threshold Evaluation

We study the effect of the rebinding threshold θ under user mobility on the Chinanet topology. In our control logic, rebinding triggers when the expected QoE improvement exceeds θ. This experiment quantifies how the threshold governs quality and overhead. Users move randomly over a one-hour trace. Fig. 5 plots QoE loss over time with $\theta = 5\%$ (blue), without rebinding (orange), and the rebinding instants (green, dashed). Rebinding suppresses the accumulation of QoE loss and periodically returns the system to a low-loss state, whereas the no-rebinding curve drifts upward and remains elevated.

We then sweep $\theta \in \{0.01, \ldots, 0.08\}$ and record, within one hour, the number of rebindings and the average QoE.

Results: According to Table VII, there is a clear trade-off between quality and overhead. Lower thresholds improve QoE but trigger more rebindings. Lowering θ from 0.05 to 0.01 reduces average QoE by 3.5% (0.1975→0.1908), yet increases rebindings 16.2× (5→81). Conversely, relaxing to 0.06 cuts rebindings 5× (5→1) while degrading QoE by 2.2% (0.1975→0.2018). A practical knee occurs at $\theta = 0.05$: relative to $\theta = 0.04$, rebinding count drops from 14 to 5 ($\approx 64\%$ fewer) for only a 1.3% QoE increase (0.1949→0.1975). In practice, θ should therefore be exposed as a tunable policy knob rather than a fixed constant. Given a target budget on rebinding operations and an acceptable QoE loss relative to the best attainable configuration, operators can select θ by inspecting trade-off curves. Operators in more QoE-sensitive environments may choose a smaller θ (accepting more rebindings), whereas those prioritizing lower control-plane overhead may choose a slightly larger θ with marginal QoE degradation.

F. Case Study: GeoLife

To understand the performance under real-world user traces, we incorporated a real-world mobility dataset, GeoLife [21], [22], in our experiments. GeoLife contains GPS trajectories from 182 users. We paired these traces with the Chinanet topology from the Internet Topology Zoo. Because raw GPS coordinates do not directly indicate transitions between network nodes, we initialized each user at a random node. We simulate a node change whenever the displacement between consecutive GPS samples exceeds 100 m, a conservative proxy for access-point scale. This procedure yields realistic mobility while maintaining a consistent mapping from geographic movement to network-level handovers.

Fig. 6 reports QoE cost over time. Our method achieves the lowest cost and the smallest temporal variance, demonstrating effectiveness and stability under real mobility. By contrast, Gateway (nearest assignment) exhibits large oscillations, as boundary crossings frequently flip assignments. The Convex baseline performs well for the initial snapshot. However, as users move, its allocation becomes misaligned with the evolving demand, and its realized QoE can even fall below Random. The Dynamic Program shows middling performance with moderate variability. MILP remains competitive and reasonably stable, yet is consistently inferior to our method across snapshots. These results on a real-world mobility dataset and an operational-scale topology corroborate that our architecture is effective and stable in practical, dynamic settings.

VI. Discussion

In this section, we briefly discuss several limitations of the current design and outline directions for future work.

Dynamic Topology: Our model assumes a fixed network topology. In networks with frequent reconfigurations, such as SDN-driven deployments, the placement and assignment modules would need to be extended to incrementally adapt to topology updates, for example, via event-driven or controller-triggered re-computation.

Flow Resource Model: The current formulation assumes uniform resource demand per flow. In practice, traffic is heterogeneous. A natural extension is to introduce flow-class–specific weights into the load-balancing objective, enabling the SDP framework to treat different traffic classes according to their QoS and security requirements.

Multiple Services: Our system currently assumes a centralized SDP control plane and a single backend instance per service. In very large-scale networks, multiple controllers and replicated service instances may coexist. Extending the placement framework to support distributed control points and flexible binding across multiple service instances would move towards a multi-service SDP fabric with improved scalability and resilience.

VII. Conclusion and Future Work

In this paper, we presented a lightweight framework for SDP gateway placement and online rebinding that adapts to user mobility. The approach combines a `PathHeatMap`-guided placement with a bi-objective score that balances delay reduction against load variability, yielding high-quality solutions. To capture mobility, we leverage RADIUS association records to build per-epoch snapshots of user locations. A deferred rebinding module then updates bindings only when a measurable QoE gain is expected. We further showed that authentication delay fits naturally into the same objective by treating the controller as a service endpoint, so improvements in propagation translate directly into lower authentication latency. Evaluation indicates that the method attains Pareto-front operating points while delivering order-of-magnitude runtime savings on the largest tested graphs. It scales smoothly with network size and demand and remains robust to the choice of weights and thresholds. In future work, we plan to prototype adapters for production SDP platforms, extend the framework to distributed and multi-domain settings, and conduct at-scale trials in live deployments. These directions are expected to further strengthen the practical applicability of SDP placement under mobility.

References

[1] R. Scott, B. Oliver, M. Stu, and C. Sean, "Zero trust architecture," *NIST Special Publication*, vol. 800, 2020, Art. no. 207.

[2] "Zero trust security market size, share & trends analysis report by deployment," 2024, Accessed: Apr. 2025. [Online]. Available: https://www.grandviewresearch.com/industry-analysis/zero-trust-security-market-report

[3] "Gartner survey reveals 63% of organizations worldwide have implemented a zero-trust strategy," 2024, Accessed: 2025. [Online]. Available: https://www.gartner.com/en/newsroom/press-releases/2024-04-22-gartner-survey-reveals-63-percent-of-organizations-worldwide-have-implemented-a-zero-trust-strategy

[4] P. Kumar, A. Moubayed, A. Refaey, A. Shami, and J. Koilpillai, "Performance analysis of SDP for secure internal enterprises," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2019, pp. 1–6.

[5] J. Singh, A. Refaey, and A. Shami, "Multilevel security framework for NFV based on software defined perimeter," *IEEE Netw.*, vol. 34, no. 5, pp. 114–119, Sep./Oct. 2020.

[6] A. Moubayed, A. Refaey, and A. Shami, "Software-defined perimeter (SDP): State of the art secure solution for modern networks," *IEEE Netw.*, vol. 33, no. 5, pp. 226–233, Sep./Oct. 2019.

[7] Y. Bello, A. R. Hussein, M. Ulema, and J. Koilpillai, "On sustained zero trust conceptualization security for mobile core networks in 5G and beyond," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 2, pp. 1876–1889, Jun. 2022.

[8] Z. Abdelhay, Y. Bello, and A. Refaey, "Toward zero-trust 6GC: A software defined perimeter approach with dynamic moving target defense mechanism," *IEEE Wireless Commun.*, vol. 31, no. 2, pp. 74–80, Apr. 2024.

[9] Y.-C. Chen, J. Kurose, and D. Towsley, "A mixed queueing network model of mobility in a campus wireless network," in *Proc. 31st IEEE Int. Conf. Comput. Commun.*, 2012, pp. 2656–2660.

[10] Y. Zhang, "User mobility from the view of cellular data networks," in *Proc. 33rd IEEE Int. Conf. Comput. Commun.*, 2014, pp. 1348–1356.

[11] Y. Liu, Z. Su, H. Peng, Y. Xiang, W. Wang, and R. Li, "Zero trust-based mobile network security architecture," *IEEE Wireless Commun.*, vol. 31, no. 2, pp. 82–88, Apr. 2024.

[12] H. Sedjelmaci, K. Tourki, and N. Ansari, "Enabling 6G security: The synergy of zero trust architecture and artificial intelligence," *IEEE Netw.*, vol. 38, no. 3, pp. 171–177, May 2024.

[13] X. Chen, W. Feng, N. Ge, and Y. Zhang, "Zero trust architecture for 6G security," *IEEE Netw.*, vol. 38, no. 4, pp. 224–232, Jul. 2024.

[14] X. Feng and S. Hu, "Cyber-physical zero trust architecture for industrial cyber-physical systems," *IEEE Trans. Ind. Cyber- Phys. Syst.*, vol. 1, pp. 394–405, 2023.

[15] W. Lei, Z. Pang, H. Wen, W. Hou, and W. Li, "Physical layer enhanced zero-trust security for wireless industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 20, no. 3, pp. 4327–4336, Mar. 2024.

[16] S. M. Nagarajan, G. G. Devarajan, M. S. Thangakrishnan, T. V. Ramana, A. K. Bashir, and A. A. AlZubi, "Artificial intelligence-based zero trust security approach for consumer industry," *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 5411–5418, Aug. 2024.

[17] Gurobi Optimization, *LLC*, "Gurobi optimizer reference manual," 2024. Accessed: May 2025. [Online]. Available: https://www.gurobi.com

[18] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," 2012, Accessed: May 2025. [Online]. Available: https://cvxr.com/cvx

[19] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[20] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The Internet topology zoo," *IEEE J. Sel. Areas Commun.*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.

[21] Y. Zheng, L. Zhang, X. Xie, and W. Ma, "Mining interesting locations and travel sequences from GPS trajectories," in *Proc. 18th Int. Conf. World Wide Web*, 2009, pp. 791–800.

[22] Y. Zheng, X. Xie, and W. Ma, "GeoLife: A collaborative social networking service among user, location and trajectory," *IEEE Data Eng. Bull.*, vol. 33, no. 2, pp. 32–39, Feb. 2010.

[23] FreeRADIUS Development Team, "FreeRadius: The World's most popular radius server," 2025. Accessed: Nov. 4, 2025. [Online]. Available: https://freeradius.org

[24] H. Sedjelmaci and N. Ansari, "Zero trust architecture empowered attack detection framework to secure 6G edge computing," *IEEE Netw.*, vol. 38, no. 1, pp. 196–202, Jan. 2024.

[25] G. R. da Silva and A. L. dos Santos, "Adaptive access control for smart homes supported by zero trust for user actions," *IEEE Trans. Netw. Service Manag.*, early access, Nov. 12, 2024, doi: 10.1109/TNSM.2024.3492379.

[26] Z. Ye et al., "User behavior-based dynamic authentication design for enhanced identity security," in *Proc. IEEE Int. Commun. Conf.*, 2025, pp. 1–6.

[27] L. Meng, D. Huang, J. An, X. Zhou, and F. Lin, "A continuous authentication protocol without trust authority for zero trust architecture," *China Commun.*, vol. 19, no. 8, pp. 198–213, 2022.

[28] W. Jing, L. Peng, H. Fu, and A. Hu, "An authentication mechanism based on zero trust with radio frequency fingerprint for Internet of Things networks," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23683–23698, Jul. 2024.

[29] Y. Bello and A. R. Hussein, "Dynamic policy decision/enforcement security zoning through stochastic games and meta learning," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 1, pp. 807–821, Feb. 2025.

[30] Y. Ge and Q. Zhu, "GAZETA: Game-theoretic zero-trust authentication for defense against lateral movement in 5G IoT networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 540–554, 2024.

[31] K. Routray and P. Bera, "Risk-aware lightweight data access control for cloud-assisted IIoT: A zero-trust approach," in *Proc. ACM Special Int. Group Data Commun. Workshop Zero Trust Archit. Next Gener. Commun.*, 2024, pp. 40–42.

[32] R. Mukta, S. Pal, M. Hitchens, H.-Y. Paik, and S. S. Kanhere, "Poster: Zero trust driven architecture for blockchain-based access control delegation," in *Proc. ACM Special Int. Group Data Commun.: Posters Demos*, 2024, pp. 48–50.

[33] C. Zanasi, S. Russo, and M. Colajanni, "Flexible zero trust architecture for the cybersecurity of industrial IoT infrastructures," *Ad Hoc Netw.*, vol. 156, 2024, Art. no. 103414.

[34] Q. Shen and Y. Shen, "Endpoint security reinforcement via integrated zero-trust systems: A collaborative approach," *Comput. Secur.*, vol. 136, 2024, Art. no. 103537.

[35] S. Hong, L. Xu, J. Huang, H. Li, H. Hu, and G. Gu, "SysFlow: Toward a programmable zero trust framework for system security," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2794–2809, 2023.

[36] H. Zhu, X. Xue, M. Xu, B.-G. Kim, X. Lyu, and S. Rani, "Zero-trust blockchain-enabled secure next-generation healthcare communication network," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 4, pp. 3201–3212, Aug. 2025.

[37] R. Dubin, "Content disarm and reconstruction of RTF files a zero file trust methodology," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1461–1472, 2023.

[38] Y. Liu et al., "Secure and scalable cross-domain data sharing in zero-trust cloud-edge-end environment based on sharding blockchain," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2603–2618, Jul./Aug. 2024.

[39] Y. Liu et al., "A blockchain-based decentralized, fair and authenticated information sharing scheme in zero trust internet-of-things," *IEEE Trans. Comput.*, vol. 72, no. 2, pp. 501–512, Feb. 2023.

[40] Z. Liu et al., "Secure edge server placement with non-cooperative game for internet of vehicles in web 3.0," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4020–4031, Sep./Oct. 2024.

[41] H. Sfaxi, I. Lahyani, S. Yangui, and M. Torjmen, "Latency-aware and proactive service placement for edge computing," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 4, pp. 4243–4254, Aug. 2024.

[42] K. Ray, A. Banerjee, and N. C. Narendra, "Learning-based microservice placement and migration for multi-access edge computing," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1969–1982, Feb. 2024.

[43] T. He, A. N. Toosi, and R. Buyya, "Efficient large-scale multiple migration planning and scheduling in SDN-enabled edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 6667–6680, Jun. 2024.

[44] X. Huang, B. Zhang, G. Ji, and C. Li, "Service coalition based joint application deployment and task assignment for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 7007–7018, May 2024.

[45] J. Li et al., "AoI-Aware user service satisfaction enhancement in digital twin-empowered edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1677–1690, Apr. 2024.

[46] S. S. Shinde and D. Tarchi, "Multi-time-Scale Markov decision process for joint service placement, network selection, and computation offloading in aerial IoV scenarios," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5364–5379, Nov./Dec. 2024.

[47] L. Yu, X. Sun, S. Shao, Y. Chen, and R. Albelaihi, "Backhaul-aware drone base station placement and resource management for FSO-Based drone-assisted mobile networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1659–1668, May/Jun. 2023.

[48] Y. Jia, C. Jin, Q. Li, X. Liu, and X. Jin, "FaaSPR: Latency-oriented placement and routing optimization for serverless workflow processing," *IEEE Trans. Netwo.*, vol. 33, no. 4, pp. 2063–2078, Aug. 2025.

[49] D. Li, P. Hong, K. Xue, and J. Pei, "Availability aware VNF deployment in datacenter through shared redundancy and multi-tenancy," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 4, pp. 1651–1664, Dec. 2019.

[50] J. Pei, P. Hong, M. Pan, J. Liu, and J. Zhou, "Optimal VNF placement via deep reinforcement learning in SDN/NFV-enabled networks," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 263–278, Feb. 2020.

[51] S. Wang, R. Urgaonkar, M. Zafer, and K. K. Leung, "Dynamic service migration in mobile edge computing based on Markov decision process," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1272–1285, Jun. 2019.

[52] S. Zheng, Z. Ren, W. C. Cheng, and H. Zhang, "Performance analysis for subgraph isomorphism based embedding service function chains," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3099–3113, Nov./Dec.– 2023.

[53] N. Promwongsa, A. Ebrahimzadeh, R. H. Glitho, and N. Crespi, "Joint VNF placement and scheduling for latency-sensitive services," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2432–2449, Jul./Aug. 2022.

[54] X. Gao, R. Liu, A. Kaushik, and H. Zhang, "Dynamic resource allocation for virtual network function placement in satellite edge clouds," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2252–2265, Jul./Aug. 2022.

[55] E. Tohidi, S. Parsaeefard, M. A. Maddah-Ali, B. H. Khalaj, and A. Leon-Garcia, "Near-optimal robust virtual controller placement in 5G software defined networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1687–1697, Apr.-Jun. 2021.

[56] R. Ward and B. Beyer, "Beyondcorp: A new approach to enterprise security," *Mag. USENIX SAGE*, vol. 39, no. 6, pp. 6–11, 2014.

[57] S. Hong, R. Baykov, L. Xu, S. Nadimpalli, and G. Gu, "Towards SDN-defined programmable BYOD (bring your own device) security," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–15.

[58] H. Wang, A. Srivastava, L. Xu, S. Hong, and G. Gu, "Bring your own controller: Enabling tenant-defined SDN apps in IaaS clouds," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2017, pp. 1–9.

[59] B. Sonkoly, J. Czentye, M. Szalay, B. Németh, and L. Toka, "Survey on placement methods in the edge and beyond," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2590–2629, Fourth Quarter 2021.

[60] R. Mijumbi, J. Serrat, J.-L. Gorricho, N. Bouten, F. De Turck, and R. Boutaba, "Network function virtualization: State-of-the-Art and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 236–262, First Quarter 2016.

[61] F. Z. Moraiset al., "PlaceRAN: Optimal placement of virtualized network functions in beyond 5G radio access networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5434–5448, Sep. 2023.

[62] B. Gao, Z. Zhou, F. Liu, F. Xu, and B. Li, "An online framework for joint network selection and service placement in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 11, pp. 3836–3851, Nov. 2022.

[63] J. Sabzehali, V. K. Shah, Q. Fan, B. Choudhury, L. Liu, and J. H. Reed, "Optimizing number, placement, and backhaul connectivity of multi-UAV networks," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21548–21560, Nov. 2022.

[64] M. F. Bari, S. R. Chowdhury, R. Ahmed, R. Boutaba, and O. C. M. B. Duarte, "Orchestrating virtualized network functions," *IEEE Trans. Netw. Service Manag.*, vol. 13, no. 4, pp. 725–739, Dec. 2016.

[65] Y. Qu et al., "Collaborative service provisioning for UAV-assisted mobile edge computing," *Chin. J. Electron.*, vol. 33, no. 6, pp. 1504–1514, 2024.

[66] K. Alizadeh Noghani, A. Kassler, J. Taheri, P. Öhlén, and C. Curescu, "Multiobjective genetic algorithm for fast service function chain reconfiguration," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3501–3516, Sep. 2023.

[67] W. Rankothge, F. Le, A. Russo, and J. Lobo, "Optimizing resource allocation for virtualized network functions in a cloud center using genetic algorithms," *IEEE Trans. Netw. Service Manag.*, vol. 14, no. 2, pp. 343–356, Jun. 2017.

[68] B. Bahrami, M. R. Khayyambashi, and S. Mirjalili, "Multi-objective placement of edge servers in MEC environment using a hybrid algorithm based on NSGA-II and MOPSO," *IEEE Internet Things J.*, vol. 11, no. 18, pp. 29819–29837, Sep. 2024.

[69] S. Yang, R. Chen, L. Cui, and X. Chang, "Intelligent segment routing: Toward load balancing with limited control overheads," *Big Data Mining Analytics*, vol. 6, no. 1, pp. 55–71, 2023.

[70] J. Luo, J. Song, F.-C. Zheng, L. Gao, and T. Wang, "User-centric UAV deployment and content placement in cache-enabled Multi-UAV networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5656–5660, May 2022.

[71] F. Wu et al., "Multi-variate time series prediction of traffic and users for dynamic RRH-BBU mapping in C-RAN," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10557–10572, Oct. 2025.

[72] H. Abdi, "Coefficient of variation," *Encyclopedia Res. Des.*, vol. 1, no. 5, pp. 169–171, 2010.

[73] N. Alon, B. Awerbuch, and Y. Azar, "The online set cover problem," in *Proc. 35th Annu. ACM Symp. Theory Comput.*, 2003, pp. 100–105.

[74] C. H. Papadimitriou and K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*. Chelmsford, MA, USA: Courier Corporation, 1998.

[75] C. Rigney, "Radius accounting," 2000, RFC 2866, Accessed: May, 2025. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc2866

[76] "PYMYSQL," 2023, Accessed: May 2025. [Online]. Available: https://pymysql.readthedocs.io

[77] X. Huang et al., "You can obfuscate, but you cannot hide: CrossPoint attacks against network topology obfuscation," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 5735–5750.
[78] X. Huang et al., "SpiderNet: Enabling BoT identification in network topology obfuscation against link flooding attacks," *IEEE/ACM Trans. Netw.*, vol. 33, no. 1, pp. 99–113, Feb. 2025.

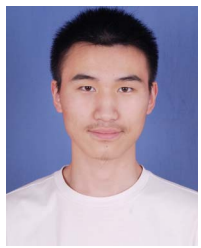
Xuanbo Huang (Member, IEEE) received the B.S. degree in the major of information security from the School of Cyber Science and Technology, University of Science and Technology of China (USTC), Hefei, China, in 2020. He is currently working toward the Ph.D. degree in the major of information security from the School of Cyber Science and Technology, USTC. His research interests include security issues in software-defined networking and programmable switch.

Kaiping Xue (Senior Member, IEEE) received the bachelor's degree from the Department of Information Security, University of Science and Technology of China (USTC), Hefei, China, in 2003, and the Ph.D. degree from the Department of Electronic Engineering and Information Science, USTC, in 2007. From 2012 to 2013, he was a Postdoctoral Researcher with the Department of Electrical and Computer Engineering, University of Florida, Gainesville, FL, USA. He is currently a Professor with the School of Cyber Science and Technology, USTC, where he is also the Director with Network and Information Center. His research interests include next-generation Internet architecture design, transmission optimization, and network security. His work was the recipient of the Best Paper awards in IEEE MSN 2017 and IEEE HotICN 2019, IEEE ICC 2025, the Best Paper Honorable Mention in ACM CCS 2022, Best Paper Runner-Up Award in IEEE MASS 2018, and Best Track Paper in MSN 2020. He is also on the Editorial Board of several journals, including the IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, and IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT. He was the Guest Editor for many reputed journals/magazines, and he was also a Track/Symposium Chair for some reputed conferences. He is an IET fellow.

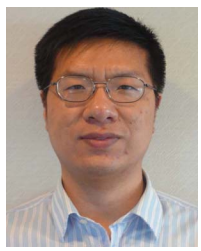
Lutong Chen (Member, IEEE) received the bachelor's and doctor's degree in the major of information security from the School of Cyber Science and Technology, University of Science and Technology of China, Hefei, China, in 2020 and 2024, respectively. He is currently an Engineer with the Network and Information Center, University of Science and Technology of China. His research interests include network security and quantum networks.

Zixu Huang (Graduate Student Member, IEEE) received the bachelor's degree in computer science and technology in 2021 from the School of the Gifted Young, University of Science and Technology of China (USTC), Hefei, China, where he is currently working toward the Ph.D. degree in the major of information security with the School of Cyber Science and Technology. His research interests include addressing network threats using software-defined networking and programmable switch.

Jiangping Han (Member, IEEE) received the bachelor's and doctor's degrees from the Department of Electronic Engineering and Information Science (EEIS), University of Science and Technology of China (USTC), Hefei, China, in 2016 and 2021, respectively. From 2019 to 2021, she was a Visiting Scholar with the School of Computing, Informatics, and Decision Systems Engineering, Arizona State University, Tempe, AZ, USA. From 2021 to 2023, she was a Postdoctoral Researcher with the School of Cyber Science and Technology, USTC, where she is currently an Associate Researcher with the School of Cyber Science and Technology. Her research interests include future Internet architecture design and transmission optimization.



Jian Li (Senior Member, IEEE) received the bachelor's degree from the Department of Electronics and Information Engineering, Anhui University, Hefei, China, in 2015, and the Ph.D. degree from the Department of Electronic Engineering and Information Science, University of Science and Technology of China (USTC), Hefei, in 2020. From 2019 to 2020, he was a Visiting Scholar with the Department of Electronic and Computer Engineering, University of Florida, Gainesville, FL, USA. From 2020 to 2022, he was a Postdoctoral Researcher with the School of Cyber Science and Technology, USTC, where he is currently an Associate Researcher with the School of Cyber Science and Technology. His research interests include quantum networking, wireless networks and network security. He is also the Editor of IEEE TRANSACTIONS ON COMMUNICATIONS and *China Communications*, and was Technical Program Committee Co-Chair of IEEE IWCMC 2025 QNC Symposium. He was the recipient of IEEE ComSoc Internet Technical Committee Early Career Award 2025.



Ruidong Li (Senior Member, IEEE) received the bachelor's degree in engineering from Zhejiang University, Hangzhou, China, in 2001, and the Doctorate of Engineering degree from the University of Tsukuba, Tsukuba, Japan, in 2008. He was a Senior Researcher with the Network System Research Institute, National Institute of Information and Communications Technology (NICT). He is currently an Associate Professor with the College of Science and Engineering, Kanazawa University, Kanazawa, Japan. He is also the Founder and Chair of the IEEE SIG on Big Data intelligent networking and the IEEE SIG on intelligent Internet edge and the Secretary of the IEEE Internet Technical Committee. He is the Chair for conferences and workshops, such as IWQoS 2021, MSN 2020, BRAINS 2020, ICC 2021 NMIC Symposium, ICCN 2019/2020, NMIC 2019/2020, and organizes the special issues for the leading magazines and journals, such as *IEEE Communications Magazine*, IEEE NETWORK, and IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING (TNSE). His research interests include future networks, Big Data networking, blockchain, information-centric networks, the Internet of Things, network security, wireless networks, and quantum Internet. He is a Member of the IEICE.